Building Popy: a download-quarantine extension grounded in OPFS

Popy is viable as a lightweight, Chromium-only MV3 extension, and the architecture you've sketched is correct — but **the `_popy` rename is cosmetic; OPFS isolation is where all real security lives**, and **a re-fetch pipeline will fail on 20–40% of real-world downloads**, so the product's correctness hinges entirely on a disciplined fallback path. This guide assumes you know extensions cold; it focuses on the parts that will bite you in production: the download-interception race, signed/POST/blob URLs, MV3 service-worker death, OPFS sandbox boundaries, and Chrome Web Store review for an extension that touches every download. Expect an MVP in ~6–10 weeks solo, dominated by interception edge cases rather than UI.

The rest of the report: an honest threat model, the exact MV3 + OPFS event flow with code, a taxonomy of every known re-fetch failure mode, fallback via `onDeterminingFilename`, CORS/auth/cookie nuances, quota and release UX, tooling and distribution, and a prioritized MVP roadmap.

---

1. A realistic threat model — what Popy actually does for you

Popy is, architecturally, a **pre-execution quarantine** for files that arrive through the browser's download API. It is not an antivirus, not a sandbox for running content, and not a defense against attacks that live inside the browser process. Framed that way, its value is real but narrow.

Meaningfully mitigated

The strongest class of threats Popy neutralizes are **passive OS-level parsers that run on any file sitting in the Downloads folder**. These are real — ACROS Security disclosed an **SCF file NTLM disclosure 0day in March 2025** that leaks credentials merely by viewing `Downloads/` in Explorer, with no double-click required 0patch (blog.0patch.com/2025/03/scf-file-ntlm-hash-disclosure.html). The same class covers CVE-2024-21320 (Windows Themes NTLM coercion via thumbnail), CVE-2025-21377 (URL file NTLM leak), Cyber Press CVE-2017-8464 (LNK icon parsing, Stuxnet-descended), Rapid7 CVE Details and macOS QuickLook / ImageIO CVEs like CVE-2019-8577. **If the file never lands in the user-visible filesystem, none of these indexers or thumbnail**

handlers touches it. OPFS achieves exactly that — it's a browser-managed store web.dev MDN Web Docs at `chrome-extension://<id>/` that the OS shell never enumerates. MDN Web Docs web.dev

The **FileFix class** of attacks (2024–2025 campaigns against `.scf`, `.url`, `.library-ms`) is fully defused in the OPFS path and partially defused in the fallback path — because even in fallback, the `_popy` suffix changes the final extension and Explorer's `ShellExecute` and preview-handler dispatch keys off the **last** extension. Renaming an `.scf` to `.scf_popy` means Explorer no longer fires the icon-parser on hover.

Double-click execution of `.exe`, `.msi`, `.scr`, `.hta`, `.vbs`, `.jar`, `.ps1`, `.bat`, `.lnk` is blocked in both paths: OPFS files are physically not double-clickable (they don't exist on the shell filesystem), and `_popy`-suffixed files show the "Open with…" dialog on Windows, no default handler on macOS (LaunchServices UTI lookup fails), and no `xdg-open` dispatch on Linux. For users who've trained themselves to double-click anything in Downloads, this eliminates the most common compromise vector.

Mark-of-the-Web and Gatekeeper bypasses that rely on container formats — ISO, VHD, 7z, MSI — were exploited throughout 2022–2023 (CVE-2022-44698 and its variant CVE-2023-24880, both used by the Magniber ransomware campaign; Medium BlueNoroff/Lazarus ISO campaigns). Born City Microsoft patched them only partially. Medium Popy's OPFS path sidesteps MOTW entirely by never writing to the filesystem; the fallback path keeps MOTW intact (Chrome sets Zone.Identifier on all downloads) and adds the rename as defense-in-depth.

Marginally mitigated

PDF JavaScript execution in Adobe Reader is stopped only if the user goes through Popy's release flow and chooses not to open with the system handler — in other words, only if the user cooperates with the quarantine model. If they "Release and Open," they've opted out of the defense. Popy can help here by offering **in-browser PDF.js preview from OPFS** (§8), which is a genuinely strong mitigation because PDF.js doesn't execute embedded JS by default.

Office macro documents get the same marginal treatment: the `_popy` suffix blocks double-click, but Office will happily open a `.docm` after the user renames it back. In-browser preview is not realistic for Office formats without a server round-trip.

Auto-mount DMG on macOS (CVE-2021-30657 Shlayer-class) is out of scope per your brief; for completeness, Popy's OPFS path does mitigate it because `hdiutil` never sees the file. The fallback path does not mitigate it — a `.dmg_popy` file is still a DMG to anyone who renames it.

Not mitigated — be explicit about these in your README

- **Browser-renderer RCE** (V8 typeconfusion, WebGPU/ANGLE bugs, WebAssembly JIT spray). The attack executes inside the browser process; no download occurs.
- **Drive-by attacks that never hit `chrome.downloads`** — for example, a malicious page that exploits the browser and pivots to persistence without ever triggering a download. Wikipedia Kaspersky
- **Files already in `Downloads/` before Popy was installed.** Popy has no API to retroactively scan or move them (see §3 on the OPFS sandbox boundary).
- **Supply-chain compromise of Popy itself** (stolen CWS publisher token, malicious dependency). The standard mitigations are pinned dependencies, reproducible builds, and a narrow maintainer circle — but this risk is inherent to every extension.
- **User-initiated release of a malicious file.** Popy is a speed bump; it cannot stop a user who chooses to run `setup.exe` after clicking Release.
- **Social-engineering lures** that bypass the download model entirely: copy-paste PowerShell (the "ClickFix" campaigns), The Hacker News typed credentials, OS-level phishing.

Why `_popy` is cosmetic and OPFS is the real thing

The rename is a **weak, one-layer social defense**: it prevents casual double-click and stops preview handlers that dispatch by extension, but any user — or any script — can rename it back in one step. OPFS isolation is categorically different: the file is owned by the browser's storage subsystem, scoped to the extension's origin web.dev (`chrome-extension://<id>/`), and **cannot leave OPFS except through `FileSystemWritableFileStream` into a handle returned by `showSaveFilePicker`.** MDN Web Docs +3 There is no move-out API; MDN Web Docs the WHATWG fs spec explicitly forbids cross-boundary `move()` (github.com/whatwg/fs/blob/main/proposals/MovingNonOpfsFiles.md). The OS shell, Spotlight, Windows Search, Defender's on-access scanner, OneDrive sync, none of them enumerate OPFS. RxDB +3 That's the crown jewel. Market Popy accordingly: "files never touch your real filesystem until you release them" is the honest claim; "renamed with `_popy` to prevent execution" is a minor secondary benefit.

Industry and open-source analogs

Popy sits between two existing categories. On one end, **HP Sure Click / Bromium**, **Microsoft Defender Application Guard**, and **Qubes disposable VMs** provide full hypervisor-backed isolation Whonix — vastly stronger, but require enterprise-grade

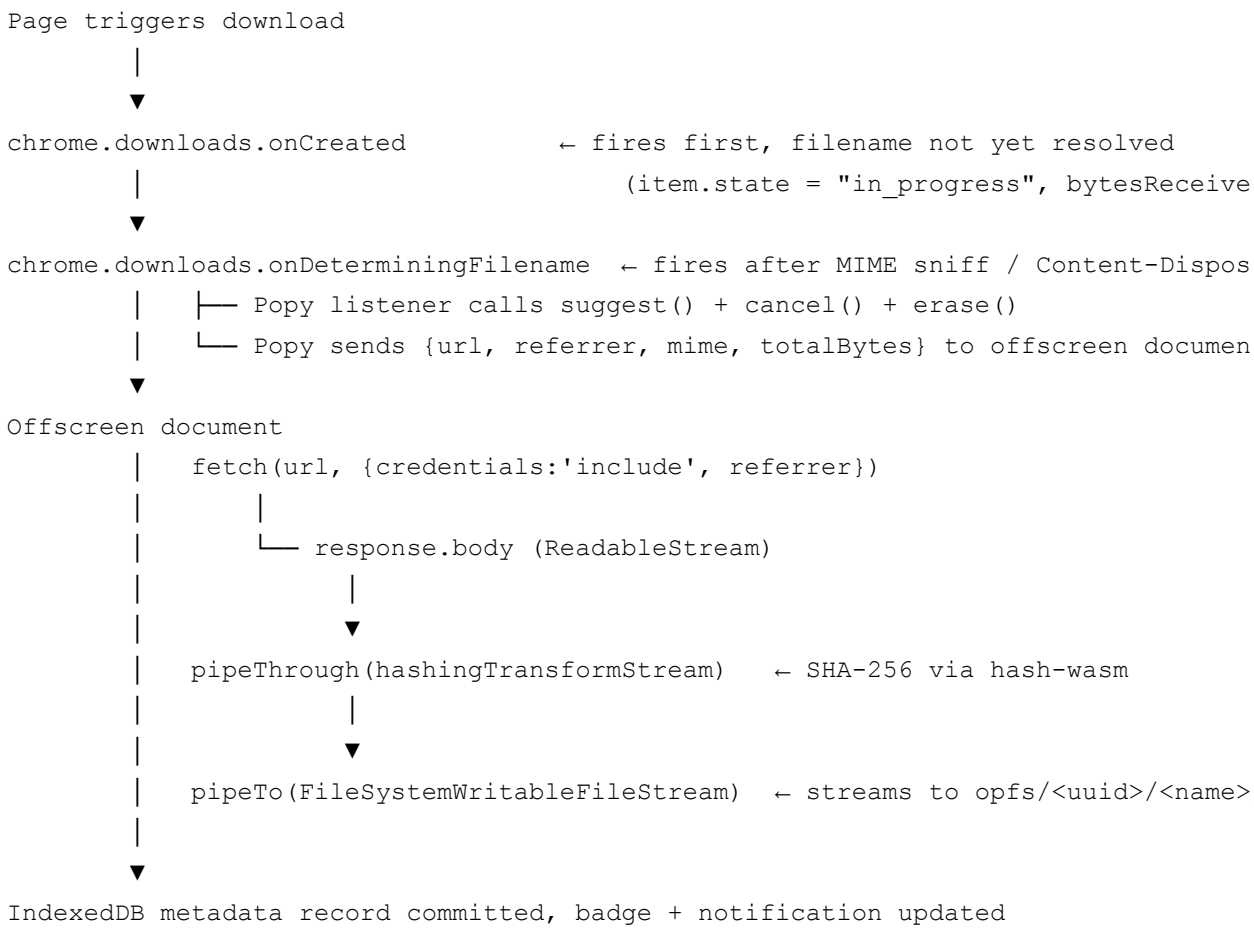
deployment. HP On the other, **Dangerzone** (Freedom of the Press Foundation, AGPLv3, github.com/freedomofpress/dangerzone) runs each document through a gVisor-sandboxed LibreOffice → pixels → PDF pipeline to produce a "safe" PDF. GitHub Freedom of the Press Foundation Dangerzone defangs content; Popy quarantines it. The natural integration — "Release into Dangerzone" — is an obvious future roadmap item.

On the extension side, there is **no meaningful prior art** for an OPFS-backed download quarantine. The OPFS-related extensions on the Chrome Web Store (OPFS Explorer Chrome Web Store by Thomas Steiner, GitHub OPFS Viewer, Chrome Web Store hasanbayatme/opfs-explorer) are DevTools panels for developers, Chrome Web Store not security tools. GitHub Malwarebytes Browser Guard focuses on URL reputation, not post-download quarantine. Malwarebytes This is an open niche.

---

2. Core architecture — the MV3 service worker + OPFS pipeline

The canonical event flow



Use `onDeterminingFilename`, not `onCreated`, as the interception hook. More metadata is

available (tentative filename, resolved MIME), and the race window before bytes hit disk is smaller. One listener per extension is allowed; `suggest()` must be called exactly once, and if you do async work after, your listener must `return true` Chrome Developers Chrome Developers (same pattern as `runtime.onMessage`). If multiple extensions register, last-installed wins Chrome Developers — this is the infamous DownThemAll coexistence hazard (github.com/downthemall/downthemall/issues/350). Document this clearly in your support page.

`chrome.downloads.cancel(id)` is racy. By the time your listener runs, Chrome already has headers and may have written hundreds of KB to a `.crdownload` temp. For a small fast file, `cancel()` may resolve with `state: "complete"` — in which case you need `chrome.downloads.removeFile({id})` + `erase({id})` to clean up. Always `await cancel()` and re-query state before proceeding.

Register listeners at top level, always

```
// background.ts — top-level, synchronous
chrome.downloads.onDeterminingFilename.addListener((item, suggest) => {
  void handleDetermining(item, suggest);
  return true;  // enables async suggest()
});
```

MV3 service workers persist only a *flag* saying "this extension is interested in this event." On wakeup, the SW script re-runs and must re-register synchronously. Any listener registered inside an async callback is silently dropped across restarts Chrome Developers (groups.google.com/a/chromium.org/g/chromium-extensions/c/hWEe4I93P_E).

The offscreen document is not optional

MV3 SWs terminate after 30 seconds idle Playwright and are hard-killed at 5 minutes wall time. Chrome Developers A multi-GB download will not survive. `chrome.runtime.Port` keepalives no longer work (developer.chrome.com/blog/longer-esw-lifetimes/). The correct pattern is an **offscreen document with `reasons: ["WORKERS"]` or `["BLOBS"]`** — offscreen docs have no lifetime cap (except `AUDIO_PLAYBACK`). Chrome Developers The SW's job becomes pure event plumbing.

```
async function ensureOffscreen(): Promise<void> {
  const url = chrome.runtime.getURL("offscreen.html");
  const ctx = await chrome.runtime.getContexts({
    contextTypes: ["OFFSCREEN_DOCUMENT"],
    documentUrls: [url],
  });
```

```
  if (ctx.length) return;
  await chrome.offscreen.createDocument({
    url: "offscreen.html",
    reasons: [chrome.offscreen.Reason.WORKERS, chrome.offscreen.Reason.BLOBS],
    justification: "Long-running fetch to OPFS with streaming hash",
  });
}
```

The fetch-to-OPFS stream

```
// offscreen.ts
import { createSHA256 } from "hash-wasm";

async function fetchToOpfs(p: {
  url: string; referrer?: string; opfsPath: string; id: string;
  originalFilename: string; totalBytes: number; mime: string;
}) {
  const root = await navigator.storage.getDirectory();
  const [dirSeg, fileSeg] = p.opfsPath.split("/");
  const dir = await root.getDirectoryHandle(dirSeg, { create: true });
  const fh  = await dir.getFileHandle(fileSeg, { create: true });
  const writable = await fh.createWritable();   // atomic swap-file semantics

  const resp = await fetch(p.url, {
    credentials: "include",
    referrer: p.referrer,
  });
  if (!resp.ok || !resp.body) {
    await writable.abort("HTTP " + resp.status);
    throw new Error("HTTP " + resp.status);
  }
  if (!passesValidator(resp, p)) {          // §3 layered validator
    await writable.abort("validator"); throw new Error("refetch-invalid");
  }

  const hasher = await createSHA256(); hasher.init();
  let received = 0, lastReport = 0;
  const tap = new TransformStream<Uint8Array, Uint8Array>({
    transform(chunk, ctrl) {
      hasher.update(chunk); received += chunk.byteLength;
      const now = performance.now();
      if (now - lastReport > 250) {
        chrome.runtime.sendMessage({ type:"progress", id:p.id, received, total:p.
        lastReport = now;
      }
      ctrl.enqueue(chunk);
```

```
      },
    });

    try {
      await resp.body.pipeThrough(tap).pipeTo(writable);   // backpressure end-to-e
    } catch (e) {
      try { await writable.abort(String(e)); } catch {}
      throw e;
    }
    return { sha256: hasher.digest("hex"), bytes: received };
  }
```

Memory stays flat at tens of MB for multi-GB files because `pipeTo` propagates backpressure all the way to TCP. MDN Web Docs Don't be tempted to `await resp.arrayBuffer()` — it defeats the entire point.

SHA-256 in-stream — pick your poison

`crypto.subtle.digest` does **not** support streaming input. Options:

1. **`hash-wasm`** in a TransformStream (shown above). Hundreds of MB/s with its WASM build, zero memory overhead. This is the right answer.
2. **Post-hoc hash** by re-reading `fileHandle.getFile().stream()` after `pipeTo` completes. Second disk read; simpler code; acceptable for MVP.
3. Do not tee the response body to buffer and hash — it'll OOM on large files.

Cloudflare's non-standard `crypto.DigestStream` does not ship in Chrome; watch the WebCrypto streams proposal but don't depend on it.

State, metadata, and OPFS layout

Store metadata in **IndexedDB**, not OPFS sidecar files. IDB has indexes and is faster to query; sidecars double I/O and make eviction painful. Use the `idb` library. Schema:

```
  interface QuarantineRecord {
    id: string;          // UUID v4, primary key
    opfsPath: string;    // "<uuid>/installer.exe_popy"
    originalFilename: string;
    sourceUrl: string;
    referrer?: string;
    tabUrl?: string;
    sizeBytes: number;
    mime: string;
    sha256: string;
```

```
  status: "fetching" | "stored" | "released" | "failed";
  createdAt: number;
  releasedAt?: number;
}
// indexes: createdAt, sha256, sourceUrl, status
```

OPFS directory layout: per-file UUID subdirectory.

```
opfs-root/
  7a8f…/installer.exe_popy
  b2c1…/installer.exe_popy   ← zero collision even with same filename
```

The UUID subdir makes filename collisions impossible across concurrent downloads from different origins, and gives you a place to drop per-file resume-state or partial-hash artifacts later. Organizing by host is a footgun (illegal characters, host collisions on same filename). Organizing flat requires UUID-prefixed names; works but you lose co-location.

Use `chrome.storage.session` for in-flight `downloadId → UUID` mappings (survives SW restarts within a session), `chrome.storage.local` for user prefs, IDB for the quarantine index.

When SyncAccessHandle matters

`FileSystemSyncAccessHandle` (Chrome 102+, methods became sync in 108) RxDB is dedicated-worker-only RxDB and requires an exclusive lock. MDN Web Docs +3 It's not available in the service worker. Google Groups The offscreen document can spawn a worker with `new Worker(...)`. **For a linear fetch→disk pipeline, the async `FileSystemWritableFileStream` is sufficient** — it does the same swap-file commit, and streaming fetch is network-bound, not disk-bound. Only reach for `SyncAccessHandle` if you implement resumable parallel range downloads; MDN Web Docs skip it for MVP.

---

3. The hard problems — re-fetch failure modes

This is where Popy lives or dies. Roughly estimated, on a general consumer population: signed URLs and native fetch-OK flows are ~60–70% of downloads; the rest is a minefield. Detect early, fall back gracefully.

S3 presigned, CloudFront signed, CDN tokens

Per AWS docs, **presigned URLs are reusable within their TTL** (docs.aws.amazon.com/AmazonS3/latest/userguide/using-presigned-url.html: "You can use

the presigned URL multiple times, up to the expiration date"). Because your re-fetch happens within seconds, time-based expiry is almost never the failure. **The exception is when the underlying STS/role credentials expire** (often ~12h even if `Expires` says 7 days). CloudFront signed URLs behave the same. AWS In practice these re-fetch cleanly.

True single-use: Google Drive `drive.usercontent.google.com/download` for files over ~40 MB returns an HTML interstitial with a `confirm` token tied to a `download_warning_*` session cookie set on the originating page. Your extension SW's cookie jar doesn't have it. Docsaid **Detection**: response Content-Type is `text/html` when you expected binary. **Mitigation**: none — fall back to rename-in-place.

POST-initiated downloads

Form POST → `Content-Disposition: attachment`. `DownloadItem` does not expose the HTTP method in Chrome; the IDL only has `url`, `finalUrl`, `referrer`, `mime`, `filename`, `totalBytes`, `startTime`, `state`, `danger`. Your GET re-fetch gets 405 or an HTML form page.

Detection proactively: maintain a `webRequest.onBeforeRequest` ring buffer keyed by URL, with `method` and `requestBody`, 30-second TTL. On `onDeterminingFilename`, look up and check method. In MV3, `webRequest` is observation-only for non-policy extensions, which is exactly what you need. **Do not attempt to replay POSTs** — multipart bodies, CSRF tokens, one-time server state, and the `Origin: chrome-extension://<id>` header will torpedo it across the board. Route POST-origin downloads straight to fallback.

JS-generated `blob:` URL downloads

`const url = URL.createObjectURL(blob); a.href=url; a.click()` is the modern SPA download pattern. `DownloadItem.url` begins with `blob:`. Blob URLs resolve only in the creating document's agent cluster — your extension SW fetch returns a network error. **Detection**: `url.startsWith('blob:')`, trivially. **Mitigation**: none realistic in MVP. A MAIN-world content script that monkey-patches `URL.createObjectURL` + `HTMLAnchorElement.prototype.click` can hoist the Blob out via `postMessage`, but it breaks on `Content-Security-Policy: script-src 'self'` with trusted-types, it races the click, and modern streaming downloads (WritableStream → server) have no consolidated Blob to catch. **Skip in MVP; fall back.**

`data:` URLs

Trivially fetchable from any context. Handle as a one-line special case; stream via `response.body`, not `arrayBuffer()`, for large base64 blobs.

Custom headers, Authorization bearers, CSRF tokens

SPA fetch: `fetch('/api/file/123', { headers: { Authorization: 'Bearer …' } })`. You don't know the token. Server returns 401/403. **Detection proactively**: `webRequest.onBeforeSendHeaders` with `extraInfoSpec: ['requestHeaders', 'extraHeaders']` captures headers; Chrome Developers maintain a 30s TTL ring buffer keyed by exact URL. **Replay**: pass captured headers to `fetch(url, {headers})` for ordinary ones. The Fetch spec forbids scripts from setting `Referer`, `Origin`, `Host`, `Cookie`, `Sec-*` Chrome Developers — those require **declarativeNetRequest session rules** with `initiatorDomains: [chrome.runtime.id]` scoping (confirmed at groups.google.com/a/chromium.org/g/chromium-extensions/c/e-mGSeuUp5A):

```
const rid = nextRuleId();
await chrome.declarativeNetRequest.updateSessionRules({
  addRules: [{
    id: rid, priority: 1,
    action: { type: 'modifyHeaders', requestHeaders: [
      { header: 'Authorization', operation: 'set', value: capturedAuth },
      { header: 'Referer',       operation: 'set', value: item.referrer },
    ]},
    condition: {
      urlFilter: item.finalUrl,
      initiatorDomains: [chrome.runtime.id],
      resourceTypes: ['xmlhttprequest'],
    },
  }],
});
try { /* fetch */ } finally { await chrome.declarativeNetRequest.updateSessionRul
```

Always remove the rule in `finally`; concurrent downloads need unique IDs.

Cookies and SameSite — the trap most people miss

With `host_permissions` for the target origin and `credentials: 'include'`, cookies are sent automatically — but **`SameSite=Strict` cookies are not sent from the SW context** because the extension's site (`chrome-extension://<id>`) differs from the cookie's site. `SameSite=Lax` cookies are sent on GETs (varies by Chrome version); `SameSite=None; Secure` always sends. MDN Web Docs `HttpOnly` cookies are invisible to JS but are sent by `fetch()` automatically.

You *could* read cookies with `chrome.cookies.get` (requires `cookies` permission) and append via DNR `Cookie` modify. This works, but **you've just built a CSRF bypass**. Don't ship it in a general-purpose extension; only consider for site-specific scoping.

iframe-initiated downloads and page service workers

`DownloadItem.referrer` reflects the iframe URL, MDN Web Docs not the top-level. Hotlink-protected CDNs check Referer against the top frame → 403. Fix via DNR Referer rewrite (overlaps with the auth-replay rule; implement once). **Page-SW-synthesized responses** (the page's own service worker stitched the download from IDB/Cache) are unfixable — your re-fetch goes to the origin and gets a different response. Fall back.

Range requests, chunked, resumable

Chrome's downloader often sends `Range: bytes=0-`; most servers reply `200 OK` with the full body on a plain GET. A few CDNs return `416 Range Not Satisfiable`. Retry with `Range: bytes=0-`. Chunked transfer is transparent to `fetch()`. For multi-minute downloads, keep the offscreen doc alive (it already is) and tick the SW every 25s with `chrome.runtime.getPlatformInfo()` to prevent cascading wakeup issues.

The layered validator

Run these checks before committing OPFS bytes. Fail on the first miss:

1. **URL prefilter**: `blob:` → skip. `data:` → decode. Google Drive >40MB → skip.
2. **Status**: `status >= 400` → fail.
3. **Content-Type**: expected binary, got `text/html` → fail.
4. **Content-Length**: differs from `DownloadItem.totalBytes` by >10% → fail.
5. **Magic bytes**: buffer first 64 KB, check against MIME (PDF starts `%PDF`, PNG `\x89PNG`, ZIP `PK\x03\x04`, PE `MZ`).
6. **Final URL**: contains `login`, `signin`, `accounts.` → fail.

Layers 1–4 catch ~95% of real failures. 5–6 are defense-in-depth.

---

4. Fallback — native download with inline rename

When re-fetch won't work, let the native download proceed into a controlled subfolder with the `_popy` suffix:

```
chrome.downloads.onDeterminingFilename.addListener((item, suggest) => {
  if (shouldQuarantineInPlace(item)) {
    suggest({
      filename: `popy-quarantine/${crypto.randomUUID()}_${safe(item.filename)}_po
```

```
      conflictAction: 'uniquify',
    });
  } else {
    // try re-fetch path
  }
  return true;
});
```

Accept that bytes land on disk briefly. OS indexers and AV will see the file. But:

- MOTW / `com.apple.quarantine` is still set by Chrome — unchanged. TextSlashPlain Red Canary
- The `_popy` suffix defuses most preview-handler exploits (Explorer and LaunchServices dispatch by extension) and blocks double-click execution across all three OSes.
- Defender's on-access scanner seeing an unknown-extension file is *better* than it seeing `invoice.exe` — heuristics trigger more aggressively on unknown containers.

You cannot then pull the file back into OPFS. There is no extension API for arbitrary filesystem read; `showOpenFilePicker` requires a user gesture + document context and would defeat the point. The fallback file lives at `Downloads/popy-quarantine/<uuid>_<name>_popy` permanently, surfaced in Popy's dashboard as "quarantined on disk." Native messaging could bridge this gap, but it requires a per-platform installer and effectively turns Popy into a hybrid app — skip for v1.

Hybrid strategy — the correct default

```
1. Observe download via onDeterminingFilename.
2. If URL is blob:, data:, drive-confirm, or webRequest shows POST → fallback imm
3. Otherwise: cancel, erase, try re-fetch. Validate layers 1–6.
4. On any validator failure → re-dispatch via chrome.downloads.download({url, fil
5. On success → OPFS write, IDB metadata, notification.
```

Hiding the download UI

`chrome.downloads.setShelfEnabled(false)` was **deprecated in Chrome 117** when the shelf was replaced by the omnibox download bubble. Use `chrome.downloads.setUiOptions({enabled: false})` (Chrome 105+), Chrome Developers which requires the `"downloads.ui"` permission. Call it around the native fallback dispatch, then re-enable. Be conservative here — users notice when downloads stop showing up.

---

5. CORS, auth, and the `chrome-extension://` origin trap

Manifest. Minimum viable:

```
{
  "manifest_version": 3,
  "name": "Popy",
  "version": "0.1.0",
  "background": { "service_worker": "background.js", "type": "module" },
  "permissions": [
    "downloads",
    "downloads.ui",
    "offscreen",
    "storage",
    "notifications",
    "declarativeNetRequest",
    "webRequest"
  ],
  "host_permissions": ["<all_urls>"],
  "action": { "default_popup": "popup.html" },
  "options_page": "options.html"
}
```

`<all_urls>` is unavoidable. You can't know in advance which origins will serve downloads. Chrome Web Store will scrutinize this — plan the justification carefully (§12). LegalForge

`credentials: 'include'` on SW fetch is required; the default is `same-origin`, MDN Web Docs and your extension is cross-origin to every target. MDN Web Docs

Forbidden headers (`Origin`, `Referer`, `Host`, `Cookie`, `Sec-*`) cannot be set via `fetch({headers})`. Use `declarativeNetRequest.updateSessionRules` with `initiatorDomains: [chrome.runtime.id]` and a narrow `urlFilter`. Google Groups `resourceTypes: ['xmlhttprequest']` is correct for SW fetch.

Opaque responses (`mode: 'no-cors'`) give you 0 bytes. Never set `mode: 'no-cors'`; MDN Web Docs with host permissions you get full CORS-bypassing access anyway.

Origin header. The SW sends `Origin: chrome-extension://<id>` on CORS-triggering requests. Some strict APIs reject it; rewrite via DNR if needed, but be aware that server-side bot-detection may flag the anomaly.

---

6. OPFS quota, eviction, and incognito

Chromium quota rules (2026, web.dev/articles/storage-for-the-web): up to 60% of total disk per origin, up to 80% total. OPFS shares the quota pool with IndexedDB and Cache. MDN Web Docs RxDB **Incognito with "clear cookies on close" drops the cap to ~300 MB total** — this is the single sharpest limit to communicate.

Check at runtime:

```
const { usage, quota, usageDetails } = await navigator.storage.estimate();
// usageDetails.fileSystem reports OPFS bytes in Chromium
const free = quota - usage;
```

Before starting a fetch with known `totalBytes`, check `free > totalBytes * 1.1`. On `QuotaExceededError` during write, abort the writable (swap file discarded, original untouched), mark the record `failed`, surface a toast. RxDB

Persistence. Call `await navigator.storage.persist()` during first-run onboarding. For extension origins this typically returns `true` immediately. Without it, Chromium may evict storage under memory pressure. web.dev

Eviction policy. Do NOT auto-evict. The whole point is manual release. Provide a dashboard UI with:

- Warning banner at 80% full.
- Sort by size, delete selected.
- Per-file manual delete, bulk delete by origin.
- Optional: auto-expire files older than 90 days (opt-in, off by default).

Very large files. For files > 2 GB or > 50% of free quota, surface a prompt: "This file is large — quarantine in Popy (slower, isolated) or download normally (faster, lives in Downloads)?" Native-fallback is the pragmatic default for multi-GB installers/ISOs.

---

7. The release flow

Two entry points: the browser action popup (list of last 5–10) and the full-page dashboard (`chrome.runtime.getURL('dashboard.html')` in a new tab).

`showSaveFilePicker()` gotchas:

- Must be called from a **user gesture in a document context** (popup, options, dashboard). Cannot be called from SW or offscreen doc.

- The gesture is consumed by the first `await`. **Call `showSaveFilePicker` first, then do async work.**
- Returned handle is outside OPFS; writes go through the normal Downloads pipeline (MOTW applied, Safe Browsing scanned on close).

```
async function releaseToDisk(id: string) {
  const meta = await getMetadata(id);   // sync from cached state
  const savePicker = await window.showSaveFilePicker({
    suggestedName: meta.originalFilename.replace(/_popy$/, ""),
    types: [{ description: meta.mime || "File",
              accept: { [meta.mime || "application/octet-stream"]: [] } }],
  });
  const outStream = await savePicker.createWritable();

  const root = await navigator.storage.getDirectory();
  const [d, f] = meta.opfsPath.split("/");
  const dir = await root.getDirectoryHandle(d);
  const fh  = await dir.getFileHandle(f);
  const file = await fh.getFile();

  await file.stream().pipeTo(outStream);  // zero-buffer streaming
  await dir.removeEntry(f);
  await root.removeEntry(d, { recursive: true });
  await updateMetadata(id, { status: "released", releasedAt: Date.now() });
}
```

No `move()` out of OPFS. The WHATWG fs spec explicitly forbids cross-sandbox move. `showSaveFilePicker` + streaming write is the only path.

Metadata display per file: original URL (clickable), referrer, download timestamp (relative + absolute), size (human-readable + bytes), MIME, SHA-256 (monospace, click-to-copy), **threat-intel lookup button** that queries VirusTotal and MalwareBazaar by hash.

VirusTotal integration (v0.2, not v0.1): public API is 4 requests/min, 500/day, free API key required. Use the file-hash endpoint (`/api/v3/files/{sha256}`) — no upload, just hash lookup. MalwareBazaar's hash query is unauthenticated and free. Surface as "0/70 engines detected" with raw counts; do NOT auto-block on hits (too many false positives for uncommon legitimate files). This requires **Limited Use disclosure** in your privacy policy for CWS review.

Release-and-Open vs Release-Only. Offer both, default to Release-Only. Release-and-Open uses `chrome.downloads.download({url: blobURL, ...})` after writing — but at that point you've already defeated quarantine. For the MVP, ship Release-Only; add Release-and-Open in v1 with a confirmation dialog.

Bulk actions, search, filter. Minimum: search by filename, filter by origin, bulk release (with one picker per file — batch picker isn't in the File System Access spec), bulk delete.

---

8. In-browser viewing — the "peek without release" pattern

The single biggest UX risk is users releasing files just to peek. Solve it for the two most common cases:

PDFs: bundle PDF.js, render from an OPFS-backed blob URL in an extension page. PDF.js runs in a sandboxed iframe with scripting disabled by default — the JS-in-PDF RCE surface is effectively closed. This is the one place where Popy offers *better* viewing than the OS.

```
const file = await opfsHandle.getFile();
const blobUrl = URL.createObjectURL(file);
// render in extension page iframe with sandbox="allow-scripts allow-same-origin"
// using pdfjs-dist viewer
```

Images: same pattern, `<img src={blobUrl}>` in an extension page. Note: if the image format has a parser exploit (TIFF, WebP), the *renderer* still parses it. Accept the residual risk — Chrome's image decoder is hardened; the OS shell handler is not.

Text / JSON / source code: decode bytes in JS, render in a `<pre>` with a syntax highlighter.

Office docs, archives, executables: do not preview. The only safe path is release + external viewer (or pipe to Dangerzone, a future integration).

Revoke blob URLs when the viewer closes (`URL.revokeObjectURL`), and scope the viewer to a dedicated extension page so CSP can be locked down: `default-src 'self'; script-src 'self' 'wasm-unsafe-eval'; object-src 'none'` .

---

9. UX for passive daily use

The design brief is "install once, forget." Any friction kills adoption.

- **Badge**: show unread quarantine count on the action icon. Clear on popup open.
- **Notification** (`chrome.notifications`): one per download, titled "Quarantined: <filename>", with "Release" and "Delete" action buttons. Throttle: max 3/minute, collapse the rest into a "N more" summary.
- **Keyboard shortcut**: `chrome.commands` manifest entry, default `Alt+Shift+P` (Popy),

opens the dashboard.

- **First-run onboarding**: 3-slide page explaining what quarantine means, why some downloads fall back, and how to release. Set a `storage.local` flag so it shows once.
- **Power-user mode**: toggle to silence notifications, keep only badge.
- **Per-origin allowlist**: "always let downloads from github.com through normally." Tactical escape hatch; implement in v1.
- **Accessibility**: ARIA labels on all interactive elements, keyboard-navigable dashboard, respect `prefers-reduced-motion`.

Don't try to be clever. The extension has one job; the UI should reflect that.

---

10. Repo structure and tooling

Framework: WXT

Pick WXT (wxt.dev). The 2025–2026 consensus is clear — WXT is the new default. CRXJS sat in beta for three-plus years, hit 2.0 in June 2025 with a new maintainer team, but is still the smaller ecosystem (~3.9k stars vs WXT's larger and more active base). Plasmo's Parcel-based builds are 2–3× slower and its maintenance signal has weakened. WXT's entrypoints model (`entrypoints/background.ts` , `entrypoints/popup/` , `entrypoints/offscreen.html`) fits the multi-context nature of Popy perfectly, it uses Vite under the hood, cross-browser output is one flag, and HMR works.

Raw CRXJS is defensible if you want maximum control — this is a tiny, tightly-scoped extension and the case for no framework abstractions is not weak. But unless you're allergic to conventions, WXT saves real time.

TypeScript setup

Use `@types/chrome` (DefinitelyTyped, actively maintained) rather than `chrome-types`. `tsconfig.json` essentials:

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "ESNext",
    "moduleResolution": "Bundler",
    "lib": ["ES2023", "DOM", "WebWorker"],
    "strict": true,
```

```
    "noUncheckedIndexedAccess": true,
    "exactOptionalPropertyTypes": true,
    "types": ["chrome", "wxt/browser"],
    "paths": {
      "@/*": ["./src/*"],
      "@lib/*": ["./src/lib/*"]
    }
  }
}
```

`lib: ["DOM", "WebWorker"]` is critical — the SW sees `WebWorker` types, the popup sees `DOM`. You'll get some benign overlap warnings; live with them.

Repo layout

```
popy/
├── .github/
│   ├── workflows/{ci.yml, release.yml}
│   ├── ISSUE_TEMPLATE/
│   └── PULL_REQUEST_TEMPLATE.md
├── entrypoints/                  # WXT auto-discovered entrypoints
│   ├── background.ts             # SW — listener registration only
│   ├── offscreen.html
│   ├── offscreen.ts              # fetch + OPFS pipeline
│   ├── popup/
│   ├── options/
│   └── dashboard/
├── src/
│   ├── lib/
│   │   ├── opfs/                 # FS wrapper, directory layout
│   │   ├── interceptor/          # download event → classification → dispatch
│   │   ├── validator/            # layered re-fetch validator
│   │   ├── metadata/             # IDB schema + queries
│   │   ├── hash/                 # hash-wasm wrapper
│   │   └── dnr/                  # declarativeNetRequest session-rule helpers
│   ├── ui/                       # shared React/Preact components
│   └── types/
├── tests/
│   ├── unit/                     # vitest
│   └── e2e/                      # playwright + launchPersistentContext
├── docs/                         # VitePress
├── public/icons/
├── wxt.config.ts
├── package.json
├── tsconfig.json
├── eslint.config.js
```

```
├── LICENSE                        # Apache-2.0
├── README.md
├── SECURITY.md
├── CONTRIBUTING.md
├── CODE_OF_CONDUCT.md
├── THREAT_MODEL.md                # ship this publicly — credibility signal
└── CHANGELOG.md
```

Testing

Unit: Vitest. Mock `chrome.*` via `@types/chrome` + manual mocks or `sinon-chrome`. Test the interceptor classifier, the validator, OPFS path building, IDB queries.

E2E: Playwright with `chromium.launchPersistentContext` and `--load-extension`:

```
// tests/e2e/fixtures.ts
import { test as base, chromium, type BrowserContext } from '@playwright/test';
import path from 'path';
export const test = base.extend<{ context: BrowserContext; extensionId: string }>
  context: async ({}, use) => {
    const ext = path.join(__dirname, '..', '..', '.output', 'chrome-mv3');
    const ctx = await chromium.launchPersistentContext('', {
      channel: 'chromium',
      args: [`--disable-extensions-except=${ext}`, `--load-extension=${ext}`],
    });
    await use(ctx);
    await ctx.close();
  },
  extensionId: async ({ context }, use) => {
    let [sw] = context.serviceWorkers();
    if (!sw) sw = await context.waitForEvent('serviceworker');
    await use(sw.url().split('/')[2]);
  },
});
```

Playwright Chromium supports MV3 extensions headless with `channel: 'chromium'` (not `chrome`). Tests: visit a test page, trigger a download, assert the file appears in OPFS via the dashboard, release and assert a file dialog was called. Set up a local test server with fixtures that exercise each failure mode (presigned-expired, POST download, blob URL, custom-header required, iframe-initiated).

CI/CD

GitHub Actions workflow: lint (eslint 9 flat config) → typecheck (`tsc --noEmit`) → vitest → playwright → WXT build for chrome + edge targets → upload artifacts. Cache `~/.pnpm-`

`store` and Playwright browsers.

Publishing. Use `chrome-webstore-upload-cli` (github.com/fregante/chrome-webstore-upload-cli) for Chrome Web Store. Secrets: `CHROME_WEBSTORE_CLIENT_ID`, `CLIENT_SECRET`, `REFRESH_TOKEN`, `EXTENSION_ID`. Edge Add-ons has its own Partner Center API; separate workflow step. Brave pulls from Chrome Web Store automatically — no separate submission.

Versioning: **semantic-release + conventional commits**. A small Vite plugin or postbuild script syncs `package.json.version` → `manifest.json.version` on build.

Licensing

Apache 2.0. MIT is fine for most code; for a security tool where contributors and auditors want explicit patent-grant protection, Apache 2.0's §3 patent clause is genuinely valuable. GPL/AGPL would complicate Chrome Web Store distribution of derivative works and discourage contribution — skip. Dangerzone uses AGPLv3 because its threat model centers on derivative-service providers; that's not Popy's situation.

Community files

- **SECURITY.md**: enable GitHub private vulnerability reporting, publish a PGP key, set up a `security@` alias, promise 72-hour acknowledgement and 90-day disclosure window. This single file does more for auditor trust than any README copy.
- **CODE_OF_CONDUCT.md**: Contributor Covenant 2.1 verbatim.
- **CONTRIBUTING.md**: dev setup, `pnpm dev`, conventional commits, DCO or CLA (DCO is lighter-weight).
- **Docs: VitePress.** It's Vite-native, fast, and your build tooling is already Vite; Docusaurus (React, MDX) is heavier and unjustified for an extension's doc site.
- **THREAT_MODEL.md** as a top-level file. Almost no security extension publishes one. Doing so honestly is a massive trust signal.

---

11. Chrome Web Store submission realities

For an extension with `<all_urls>` + `downloads`, expect **enhanced review** and a 3–10 day initial review (longer than the typical 1–3 day window). Google tightened data-handling disclosures in January 2025 (developer.chrome.com/docs/webstore/program-policies/user-data-faq). Concrete requirements:

Permission justifications (filled in the Privacy tab of the Developer Dashboard):

- `downloads` : “Core functionality: Popy intercepts new downloads via `chrome.downloads.onDeterminingFilename` and redirects them to an isolated sandbox so the user can inspect files before they touch the filesystem.”
- `downloads.ui` : “Briefly suppress the download shelf during interception to avoid confusing UX where a download appears to start and disappear.”
- `storage` : “Persist user preferences and per-file metadata (source URL, timestamp, hash) in `chrome.storage.local` and IndexedDB.”
- `notifications` : “Inform the user each time a file is quarantined, with inline Release/Delete actions.”
- `offscreen` : “Host long-running fetch-to-OPFS streaming operations that exceed the 30-second service worker lifetime.”
- `declarativeNetRequest` : “Preserve `Referer` and `Origin` headers when re-fetching downloads, so hotlink-protected CDNs and origin-checking APIs return the expected bytes.”
- `webRequest` : “Observe (non-blocking) outbound request headers to correlate authentication tokens and HTTP methods with subsequent downloads, enabling correct re-fetch. No request blocking or modification is performed via this API.”
- `host_permissions: <all_urls>` : “Popy must re-fetch downloads from arbitrary origins; the user chooses which sites to download from, and we cannot predict them at install time. All fetched bytes remain in the browser’s Origin Private File System until the user explicitly releases them.”

Privacy policy is mandatory. Must disclose: no data collection, no analytics, no remote code, OPFS-local storage, optional VirusTotal hash lookup with the Limited Use statement (“The use of information received from VirusTotal will adhere to the Chrome Web Store User Data Policy, including the Limited Use requirements”). Host the policy at a stable URL.

Single-purpose statement: “Popy quarantines browser downloads in an isolated browser sandbox until the user explicitly releases them.” One sentence, narrow scope.

Gotchas: no remote code (MV3 forbids it anyway). Do not bundle minified unrecognizable code — reviewers run static analysis. Ship source maps. Mention the GitHub repo URL in the listing.

Edge Add-ons: separate submission via Partner Center; policies align closely with CWS. Review time ~1–5 days.

Brave, Vivaldi, Arc, Opera pull directly from the Chrome Web Store. No separate

submission for Brave. Opera has its own store if you want visibility there.

Self-hosted CRX with `update_url` pointing at your own `update.xml` is supported for enterprise/sideload; useful for a "canary" channel but not needed for v0.1.

---

12. MVP roadmap — be disciplined

v0.1 — public preview (weeks 1–8)

Ship these and only these:

- `onDeterminingFilename` interception, cancel + offscreen doc + re-fetch + OPFS write with streaming SHA-256.
- Layered validator (layers 1–4 only).
- Fallback via rename-in-place when URL is `blob:` , when re-fetch fails validation, or when webRequest detected POST.
- IDB metadata, UUID-per-file OPFS layout.
- Popup with last 10 files, Release / Delete / Open-URL buttons.
- Dashboard page with filter-by-origin, sort by time/size.
- Badge count + notifications with inline actions.
- First-run onboarding explaining quarantine.
- `chrome.commands` shortcut.
- Apache 2.0 license, SECURITY.md, THREAT_MODEL.md, docs site with architecture overview.

v0.2 — hardening (weeks 9–14)

- `webRequest` header capture ring buffer + DNR replay for Authorization/CSRF.
- DNR Referer/Origin rewrite on re-fetch.
- Validator layers 5–6 (magic bytes, final-URL heuristics).
- VirusTotal + MalwareBazaar hash lookup UI (opt-in).
- PDF.js in-browser preview from OPFS.
- Image preview from OPFS.

- Per-origin allowlist (“trust github.com, never quarantine”).
- Quota warning banner.

v1.0 — polish (weeks 15–20)

- Bulk actions (release/delete multiple).
- Large-file prompt (>2 GB or >50% free quota).
- Power-user silent mode.
- Internationalization scaffolding.
- Chrome Web Store + Edge Add-ons submissions with review-response playbook.
- Paid security audit (budget $5–15k from foundations like OpenSSF, or crowdfund via the extension’s users).

v2.0 — ambitions

- Native-messaging companion (Windows signed installer, macOS notarized pkg) to import existing Downloads files into OPFS. Stops being a pure extension; may warrant a separate repo.
- Dangerzone integration (file a PR upstream for a “send to Dangerzone” URL scheme).
- Enterprise policy support (managed storage, chrome.storage.managed schema).
- Firefox MV3 port (requires working around the lack of offscreen documents; nontrivial).

Realistic solo timeline

For an experienced extension developer: **6–8 weeks to v0.1**, dominated by the interception edge-case matrix (not by UI). Week 1: scaffolding + happy-path interception. Weeks 2–3: every failure mode reproduced in a test harness, fallback working. Weeks 4–5: OPFS + IDB + dashboard. Week 6: onboarding, notifications, polish. Weeks 7–8: Chrome Web Store review cycle and response.

Conclusion

Popy is a plausible product because the browser finally has the primitives — OPFS, streaming fetch, offscreen documents, DNR session rules — that make a lightweight pre-execution quarantine feasible without hypervisors or native code. But the interesting engineering isn’t the OPFS write; it’s the 40% of downloads that break on re-fetch and the

disciplined fallback that keeps the extension useful anyway. Ship the hybrid re-fetch-with-rename-fallback pipeline, be honest in the threat model about what's cosmetic versus structural, and OPFS's hard sandbox boundary gives you a real, if narrow, security benefit that no OS indexer, preview handler, or accidental double-click can breach. The product lives or dies on that boundary and on your discipline about the edge cases.

The clearest competitive moat is also the simplest: **publish THREAT_MODEL.md before you publish v0.1.** Almost no Chrome Web Store security extension does, and the few that do (uBlock Origin, Bitwarden) earn permanent user trust from it. For a one-person project trying to attract contributors and auditors, that single file is worth more than six months of marketing.